CS2003 · AI: Search Methods for Problem Solving

Traveling Salesperson Problem (TSP) Solver

Anytime Iterated Local Search — Candidate-Restricted 2-Opt

Indian Institute of Technology Madras (IIT Madras)

Student **Vidhatri HR**	Roll Number **23f2000575**
Email **23f2000575@ds.study.iitm.ac.in**	Script File **23f2000575.py**

✓ Pure Python 3 Std-Lib | ◴ 300s Wall-Clock Limit | ↻ Anytime Streaming Output

1 Overview

This report documents the design and implementation of an anytime Traveling Salesperson Problem (TSP) solver for both Euclidean and Non-Euclidean instances. The solver is engineered around three strict operational constraints — a pure Python 3 standard-library implementation, a hard 300-second wall-clock budget, and anytime streaming behaviour, where every printed tour permutation is a valid, independently evaluable solution and the last line printed is the one that counts.

The core algorithm is an Iterated Local Search (ILS): a greedy nearest-neighbor construction seeds the search, a candidate-list-restricted 2-opt local search polishes the tour to a local optimum, and random segment-reversal perturbations repeatedly kick the search out of that optimum so refinement can continue until the clock runs out.

2 Algorithmic Approach

2.1 Initial Tour Construction — Nearest Neighbor

The algorithm builds a starting tour with the greedy Nearest Neighbor heuristic, beginning at city 0 and repeatedly selecting the closest unvisited city until every city has been included.

- Anytime safety: construction completes in milliseconds (<5 ms for N=100) and is printed to stdout immediately, guaranteeing a valid baseline tour is on record before any further search begins.

2.2 Candidate Neighbor Lists

Exhaustive 2-opt evaluates every pair of edges — roughly $N^2/2$ comparisons per pass — which does not scale in interpreted Python. To keep the search fast, the solver precomputes a static candidate list before search begins:

- For every city, the top k = min(12, N−1) nearest neighbors by distance are precomputed and stored.

Traveling Salesperson Problem (TSP) Solver

Anytime Iterated Local Search — Candidate-Restricted 2-Opt

Indian Institute of Technology Madras (IIT Madras)

- During local search, reconnection moves for city a only test endpoints drawn from its candidate list, cutting per-pass complexity from O(N²) down to O(k·N) while preserving the highest-probability improving edges.

2.3 Candidate-Restricted 2-Opt Local Search

The local search systematically explores edge exchanges restricted to candidate neighbors:

- Inverse index table: a position array tracks the current tour index of every city in O(1) time.
- Move evaluation: for edge (a, b), a candidate reconnection to c in neighbors(a) at index j = position[c] is evaluated in O(1) via the delta below, where d = tour[(j + 1) mod N].
- In-place update: if Delta < −1e-9, the segment between i+1 and j is reversed in place via Python slice assignment, and the position mapping is re-synchronized.

```
Delta = (D[a][c] + D[b][d]) − (D[a][b] + D[c][d])
```

2.4 Perturbation & Iterated Local Search (ILS)

Once 2-opt reaches a local optimum, the solver runs an ILS loop:

1	**Perturbation (Kick)** Select two random indices i < j and reverse the subsegment tour[i:j].
2	**Local Search Refinement** Re-apply candidate-restricted 2-opt to the perturbed tour.
3	**Acceptance Criterion** If the refined tour is shorter than the global best, accept it as the new best and emit it to stdout immediately.
4	**Time-Bounded Execution** Repeat the cycle until the wall-clock deadline is reached.

3 Architecture & Implementation

The entire pipeline is encapsulated within the TSPSolver class in 23f2000575.py:

read_problem_from_stdin()	Reads metric type, city count N, coordinates, and distance matrix from standard input.
calculate_total_tour_length(tour)	Computes the round-trip distance for a given tour permutation.
construct_nearest_neighbor_tour(start_city)	Generates the initial greedy nearest-neighbor tour.

Traveling Salesperson Problem (TSP) Solver

Anytime Iterated Local Search — Candidate-Restricted 2-Opt

Indian Institute of Technology Madras (IIT Madras)

build_candidate_neighbor_lists()	Precomputes the k = 12 nearest neighbors for each city.
optimize_tour_with_restricted_two_opt(tour)	Executes candidate-restricted 2-opt passes until convergence or timeout.
perturb_tour_by_random_segment_reversal(tour)	Applies random subsegment reversal to escape local minima.
print_tour_to_stdout(tour)	Formats and flushes the space-separated tour to standard output.
run_iterated_local_search(initial_tour)	Coordinates the perturbation-refinement loop within the time limit.
run()	Entry point orchestrating input ingestion, baseline creation, search execution, and guaranteed final output.

4 Operational Guardrails

Safety Margin
A 5-second safety margin (safety_margin = 5.0) stops execution at T = 295s, ensuring clean output flushing and process termination within the 300s limit.

Edge Cases
Instances with N = 1 are handled immediately with constant-time emission of [0].

Anytime Output
Output is flushed on every improvement, guaranteeing the best solution found so far is always captured.

End of Report — 23f2000575.py · TSP Anytime Solver · CS2003, IIT Madras